



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G5

System Integration and Architecture

System Design Document (SDD)

Project Title: DisasterAid Connect

Prepared By: Labaya, Godwin M.

Version: 0.2

Date: 02/27/25

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/20/2026	Godwin M. Labaya	Initial draft	Draft
0.2	02/27/2026	Godwin M. Labaya	Added in final chapters	Final
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

DisasterAid Connect is an emergency, charity application where it allows users to scroll through disaster reports, know where relief distribution happens and volunteer coordination during emergencies. The system includes in Spring Boot backend API, React web application and Android mobile app, all integrated to make sure that no one misses a beat.

1.2 Objectives

1. Build a fully operational disaster response application that enables users to view real-time disaster reports, locate relief distribution centers, and participate in volunteer coordination activities.
2. Design and develop a scalable three-tier architecture consisting of:
 - Spring Boot backend (RESTful API services)
 - React web application (user interface) Android mobile application (mobile access platform).
 - Create RESTful APIs for communication between all system components
3. Design a responsive user interface that works consistently across web and mobile platforms
4. Provide mechanisms for reporting, updating, and viewing disaster incidents and relief activities in near real-time.

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Dashboard
- Disaster reports
- Profile
- Payment Integration Request
- Maps Integration

- Request help
- Payment History

Excluded Features:

- Inventory management automation
- Email notification system
- Social media login integration
- Push notifications
- Advanced analytics dashboard

1.0 INTRODUCTION

1.1 Purpose

This document provides the comprehensive system specification for DisasterAid Connect, an emergency disaster response and aid coordination platform.

It defines the system objectives, scope, functional requirements, non-functional requirements, and user interactions. This document serves as a reference for developers, stakeholders, and project supervisors to ensure clear understanding and successful implementation of the system.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: DisasterAid Connect

Domain: Disaster Response / Emergency Management

Primary Users:

1. Affected Individuals
2. Donors

Problem Statement: During disasters, communication gaps and lack of centralized information cause delays in relief distribution and volunteer coordination. Many affected individuals do not know:

- Where to request help
- Where relief centers are located
- How to connect with donors

There is a need for a centralized, accessible digital platform to coordinate disaster response efficiently.

Solution: DisasterAid Connect provides a centralized platform where:

Disaster incidents are reported and displayed

- Users can request aid
- Donors can contribute financially
- Administrators manage and verify disaster data

The system ensures consistent access across web and Android platforms.

2.2 Core User Journeys

Journey 1: Affected User Requesting Aid

1. User registers or logs in
2. Selects the button that goes to the map page
3. Mark a new point
4. Post description about current status
5. User presses confirm button

Journey 2: Donor Giving Aid

1. User logs in
2. Selects the button that goes to the map page
3. Checks the marked points manually or
4. Uses the search filter or easy access
5. Submits donation
6. Receives confirmation

Journey 3: Administrator Managing Disasters/Requests

1. Admin logs in
2. Checks if there are wrong reports of disaster/requests
3. Can either remove the pin, edit details or mute users

Journey 3: Administrator Monitoring Donations

1. Admin logs in
2. Monitors donations to make sure everything is working

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Disaster Report listing and detail view
3. Aid request submission
4. Donation/payment submission
5. Admin disaster management

SHOULD HAVE

1. Map-based disaster visualization
2. User profile management
3. Status tracking for aid requests
4. Responsive design for all screen sizes
5. Search filter

COULD HAVE

1. Basic reporting dashboard
2. User activity summary
3. Disaster severity filtering

WON'T HAVE

1. Push notifications
2. Social login
3. Advanced analytics

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot password
- **Fields (Registration):** Username, Email, Password, Confirm Password, Security Question, Security Answer

- **Fields (Login):** Email, Password
- **Validation:** Email format validation, password minimum 8 characters, password confirmation match, email uniqueness

- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT access and refresh tokens, password hashing using bcrypt (salt rounds = 12), role-based access control (USER / ADMIN), HTTPS communication

Feature: Disaster Reports

- **Screens:** Disaster Listing, Disaster Detail
- **Display:** Title, description, severity level, status, created_at
- **Search/Filter:** By severity_level or status
- **Map Integration:** Displays disaster markers using stored latitude and longitude
- **API Endpoints:** GET /disasters, GET /disasters/{id}, POST /disasters (Admin only), PUT /disasters/{id}, DELETE /disasters/{id}

Feature: Aid request

- **Screens:** Request Aid Form
- **Fields:** Disaster ID (auto-selected from map), Aid Type (Food, Medical, Shelter, Funds), description, quantity
- **Process:** User selects an active disaster from the map → User submits aid request → System stores request with default status = **Pending** → Admin can update status to Approved / Completed / Rejected
- **Status values:** Pending / Approved / Completed / Rejected
- **API Endpoints:** POST /aid/request, GET /aid/requests, PUT /aid/request/{id}/status (Admin)

Feature: Donation & Payment

- **Screens:** Donation Form, GCASH Checkout Page
- **Fields:** Disaster ID, Donation Amount, Payment Method
- **Process:** User selects disaster → Enters donation amount → Redirected to GCash checkout → If payment successful → Donation record created → Payment record stored → Confirmation displayed
- **Payment Status:** Pending / Success / Failed
- **API Endpoints:** POST /donations, GET /donations/history, POST /payments/process

Feature: Admin Panel

- **Screens:** Disaster Management, Aid Request Review, User Management, Donation Monitoring
- **Functions:** Create, update, and delete disaster reports Approve or reject aid requests Mute users (as stated in Acceptance Criteria AC-5) View donation and payment records
- **Access Control:** Admin role required (JWT with admin role)

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new user
- When I enter valid email and strong password
- And confirm password matches
- And click "Register"
- Then my account should be created
- And I should be redirected to the dashboard

AC-2: Submit Aid request

- Given I am logged in as a customer
- When I select an active disaster
- And fill in the aid request form with valid details
- And submit the request
- Then the request should be saved in the system
- And its status should be marked as "Pending"
- And I should see confirmation of submission

AC-3: Offer Financial Donation

- Given I am logged in
- When I select a pin
- And choose to donate funds
- And complete payment successfully
- Then a payment record should be created
- And I should receive donation confirmation

AC-4: Admin Approves Aid Request

- Given I am logged in as an administrator
- When I review a pending aid request

- And change its status to "Approved"
- Then the user should see the updated status

AC-5: Admin Managing False Disasters

- Given I am logged in as an administrator
- When I check the map and see a false report
- I can either change details of the request, remove the request, or mute the user

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app startup: ≤ 3 seconds
- Support at least 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication
- Password hashing with Bcrypt (salt rounds = 12)
- Role-based access control
- SQL injection prevention

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

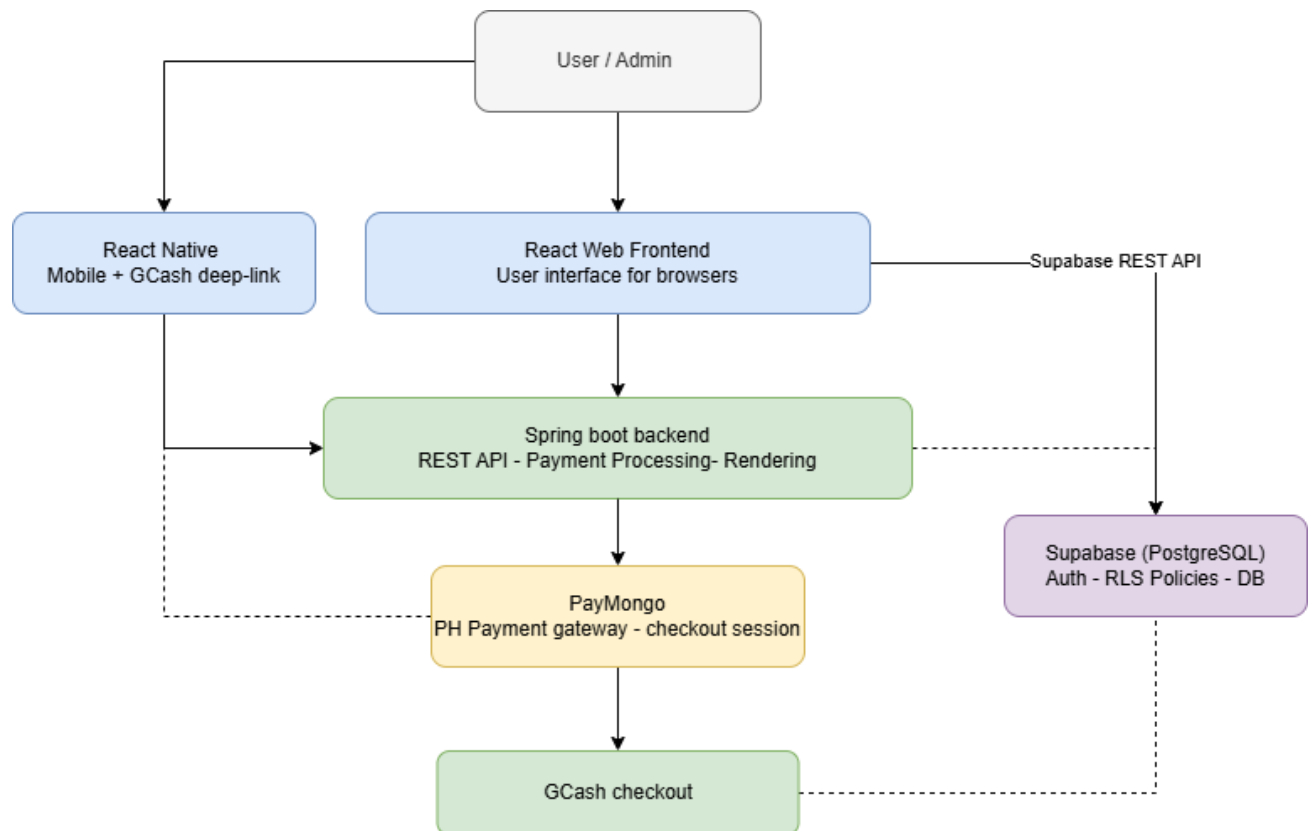
3.4 Usability Requirements

- New users can submit an aid request within 5 minutes
- Clear navigation across dashboard and disaster pages
- Consistent UI between web and mobile
- Clear validation error messages
- Mobile touch targets minimum 44x44px
- Accessible design aligned with WCAG 2.1 Level AA (web)

4.0 SYSTEM ARCHITECTURE

1.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** Native React, CSS, Axios
- **Mobile:** Kotlin
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Render (Backend/Frontend), (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

1.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
----------	---

Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	<pre>{ "success": boolean, "data": object null "error": { "code": stringm "message": string, "details": object null "role": "USER" }, "timestamp": ISO-8601 String }</pre>

1.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Register new user
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "username": "johndoe", "email": "john@email.com", "password": "Password123", "confirmPassword": "Password123", "securityQuestion": "What is your mother's maiden name?", "securityAnswer": "Smith" }</pre>

Response Structure	<pre>{ "success": true, "data": { "user": { "id": 1, "username": "johndoe", "email": "john@email.com", "role": "USER" }, "accessToken": "jwt-access-token", "refreshToken": "jwt-refresh-token" }, "error": null, "timestamp": "2026-02-27T10:30:00Z" }</pre>
---------------------------	---

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": <email>, "password": <password>, }</pre>
Response Structure	<pre>{ "success": true, "data": { "user": { "email": "user@email.com", "firstname": "John", "lastname": "Doe", "role": "USER" }, "accessToken": "jwt-access-token", "refreshToken": "jwt-refresh-token" }, "error": null, "timestamp": "2026-02-27T10:30:00Z" }</pre>

1.3 Error Handling

HTTP Status Codes

- 200 OK – Successful request
- 201 Created – Resource created
- 400 Bad Request – Invalid input
- 401 Unauthorized – Authentication required/failed
- 403 Forbidden – Insufficient permissions
- 404 Not Found – Resource doesn't exist

- 409 Conflict – Duplicate resource
- 500 Internal Server Error – Server error

Error Code Examples

Example 1	<pre> json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>

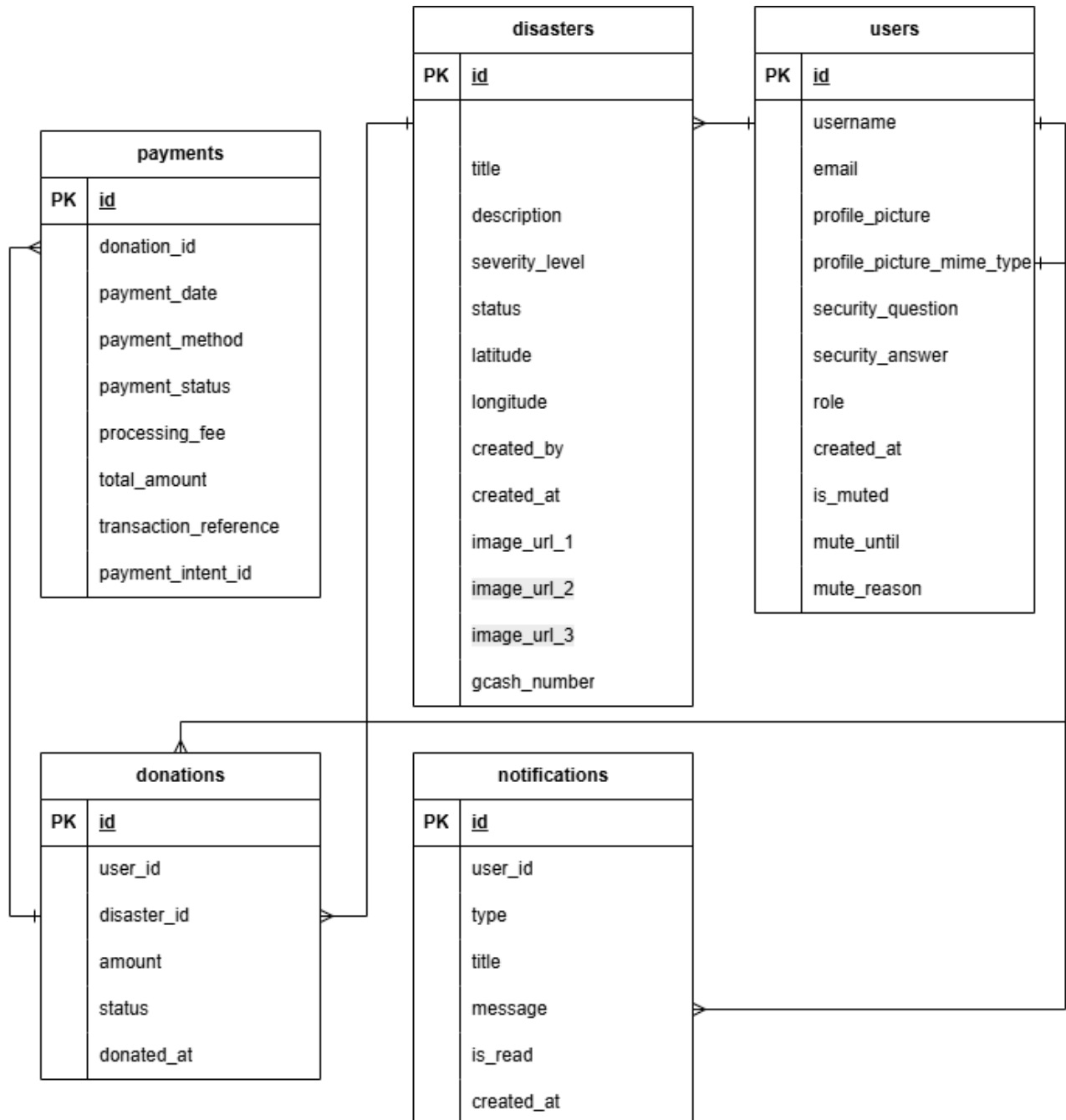
Common Error Codes

- AUTH-001 – Invalid credentials
- AUTH-002 – Token expired
- AUTH-003 – Insufficient permissions
- VALID-001 – Validation failed
- DB-001 – Resource not found
- DB-002 – Duplicate entry
- BUSINESS-001 – Invalid disaster status
- SYSTEM-001 – Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- **One-to-Many:** User → Aid requests (One user can submit multiple aid requests)
- **One-to-Many:** User → Donations (One user can donate multiple times)
- **One-to-Many:** User (Admin) → Disasters (One admin can create multiple disaster reports)
- **One-to-Many:** Disaster → Aid requests (One disaster can have multiple aid requests)
- **One-to-Many:** Disaster → Donations (One disaster can receive multiple donations)

Key Tables:

1. **users** - User accounts and role management
2. **disasters** - Disaster reports created by administrators
3. **aid_requests** - Donation requests by users
4. **donations** - Monetary contributions to disasters
5. **payments** - Stores GCASH transaction details

Table Structure Summary:

- **users:** id, username, email, password, security_question, security_answer, is_admin
- **disasters:** id, title, description, severity_level, status, created_by, created_at
- **aid_requests:** id, user_id, disaster_id, description, aid_type, quantity, status, created_at
- **donations:** id, user_id, disaster_id, amount, payment_method, transaction_id, status, donated_at
- **payments:** id, donation_id, payment_date, payment_method, payment_status, processing_fee, total_amount, transaction_reference

7.0 UI/UX DESIGN

5.1 Web Application Wireframes

Note: This should be wireframes from Figma

Dashboard

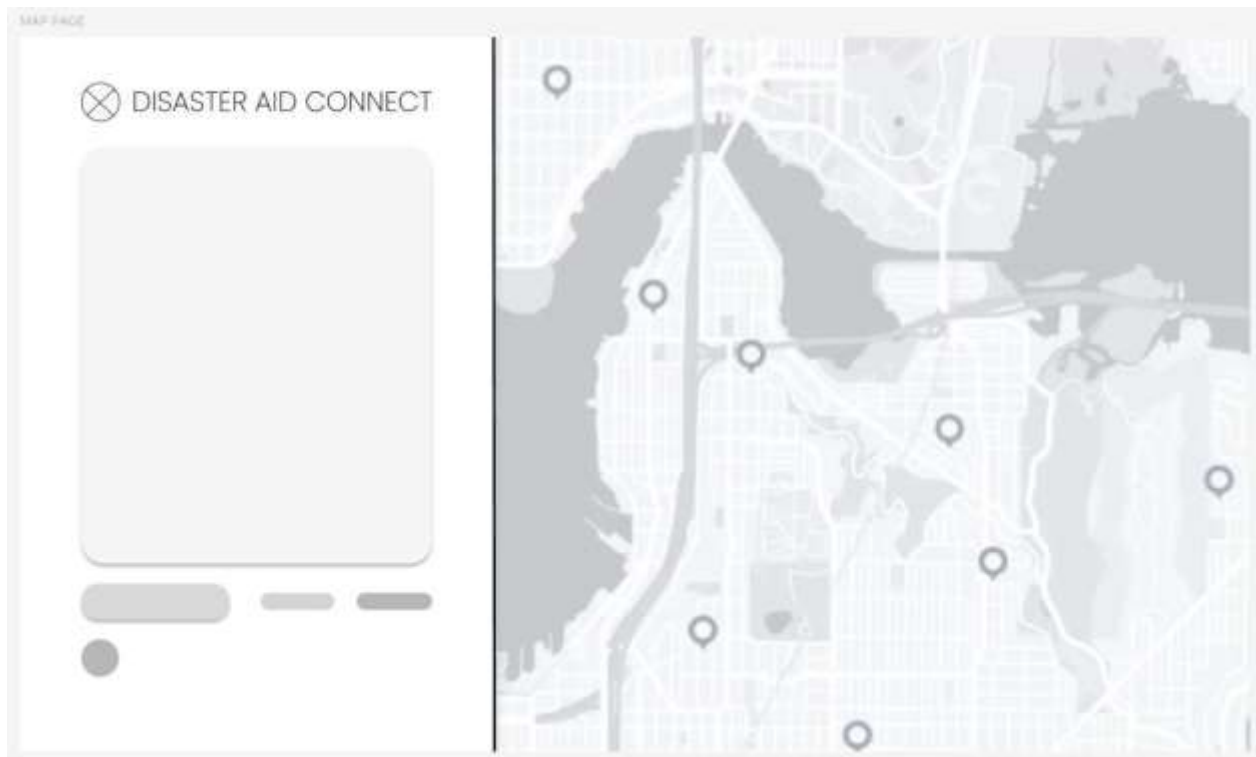


Header: [Logo] [Profile] [Logout]

Content: Stats Pages, Button to Map Page

Each Stats Pages: Donations/History, Requests/History,

Map Page Feature



Sidebar: [Logo] [Details Pane] [Back Button] [Aid Button] [Request Button] [Point Selector]

Map View (Large)

Login



Logo

Right side bar: Email address, password, Button

Left side: Short description about application

Register



Logo

Right side bar: Username, Email address, Password, Confirm Password, Security Question, Answer to Security Question, Button

Left side: Short description about application

Gcash Checkout Page



Header: [Logo] [Profile] [Logout]

Middle Pane: [GCASH LOGO] [Amount to pay] [Transaction Id] [Enter your MPIN]

5.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Dashboard



Header: [Logo] [Profile] [Logout]

Content: Stats Pages, Button to Map Page

Each Stats Pages: History, Notifications of payments, Users helped

Map Page Feature



Header: [Logo] [Profile] [Logout]

Bottom pane: [Details Pane] [Back Button] [Aid Button] [Request Button] [Point Selector]

Map View (Large)

Login



Logo

Middle pane: Email address, password, Button

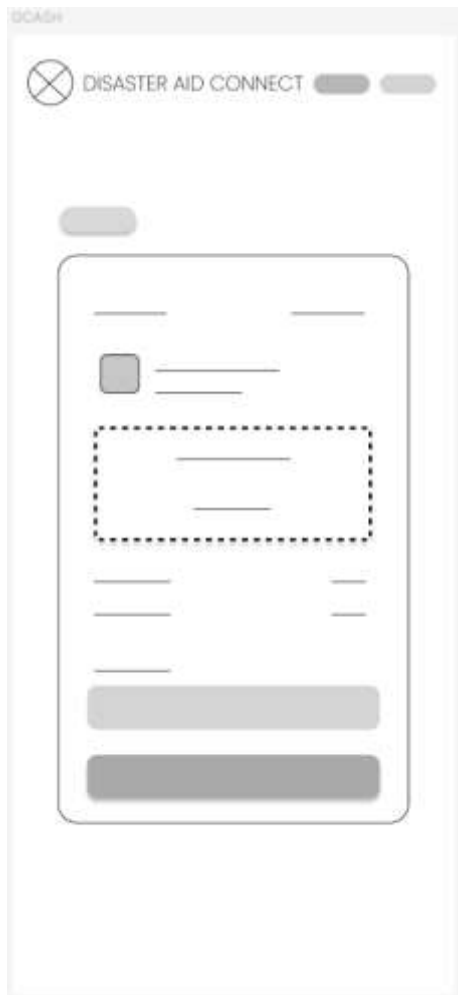
Register



Logo

Middle pane: Username, Email address, Password, Confirm Password, Security Question, Answer to Security Question, Button

Gcash Checkout Page



Header: [Logo] [Profile] [Logout]

Middle Pane: [GCASH LOGO] [Amount to pay] [Transaction Id] [Enter your MPIN]

Design System:

- **Colors:** Primary (#492828), Secondary (#656D3F), Success (#84934A), Error (#FF0000)
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Map Integration

Day 2: Request/Aid Feature

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup

Day 2: Authentication pages (login, register)

Day 3: Home page implementation

Day 4: Profile page implementation

Day 5: Reviewing of pages

Week 6: Complete Web Features/Payment Integration

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: Payment API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Request/Aid implementation

Day 4: Map Integration

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Render)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)

2. Backend API(Week 3-4)
3. Core features functionality (Week 4)
4. Payment Integration(Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements
- Focus on harder concepts than easy ones